


```

    10
0    15.3
1     NaN
2    17.8
3     NaN
4    17.8
...    ...
1007   NaN
1008  21.0
1009   NaN
1010  21.0
1011   NaN

```

```
[1012 rows x 11 columns]
```

```
raw_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1012 entries, 0 to 1011
```

```
Data columns (total 11 columns):
```

#	Column	Non-Null Count	Dtype
0	0	1012 non-null	float64
1	1	1012 non-null	float64
2	2	1012 non-null	float64
3	3	506 non-null	float64
4	4	506 non-null	float64
5	5	506 non-null	float64
6	6	506 non-null	float64
7	7	506 non-null	float64
8	8	506 non-null	float64
9	9	506 non-null	float64
10	10	506 non-null	float64

```
dtypes: float64(11)
```

```
memory usage: 87.1 KB
```

```
raw_df.isnull().sum()
```

```

0      0
1      0
2      0
3    506
4    506
5    506
6    506
7    506
8    506
9    506
10   506
dtype: int64

```

```
raw_df.describe()
```

	0	1	2	3	
count	1012.000000	1012.000000	1012.000000	506.000000	506.000000
mean	180.143778	12.008350	16.834792	0.069170	0.554695
std	188.132839	17.250728	9.912616	0.253994	0.115878
min	0.006320	0.000000	0.460000	0.000000	0.385000
25%	0.257830	0.000000	8.375000	0.000000	0.449000
50%	24.021000	7.240000	18.100000	0.000000	0.538000
75%	391.435000	16.780000	21.890000	0.000000	0.624000
max	396.900000	100.000000	50.000000	1.000000	0.871000

	5	6	7	8	9
count	506.000000	506.000000	506.000000	506.000000	506.000000
mean	6.284634	68.574901	3.795043	9.549407	408.237154
std	0.702617	28.148861	2.105710	8.707259	168.537116
min	3.561000	2.900000	1.129600	1.000000	187.000000
25%	5.885500	45.025000	2.100175	4.000000	279.000000
50%	6.208500	77.500000	3.207450	5.000000	330.000000
75%	6.623500	94.075000	5.188425	24.000000	666.000000
max	8.780000	100.000000	12.126500	24.000000	711.000000

```
raw_df.dtypes
```

0	float64
1	float64
2	float64
3	float64
4	float64
5	float64
6	float64
7	float64
8	float64

```

9      float64
10     float64
dtype: object

# Preprocess dataset
X = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]]) #
Features
y = raw_df.values[1::2, 2] # Target (House Prices)

# Convert to DataFrame
columns = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS',
'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']
df = pd.DataFrame(X, columns=columns)
df['PRICE'] = y

# Splitting Dataset
X = df.drop('PRICE', axis=1)
y = df['PRICE']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Model Training
model = LinearRegression()
model.fit(X_train, y_train)

LinearRegression()

# Prediction
y_pred = model.predict(X_test)
y_test_pred = model.predict(X_test)
y_train_pred = model.predict(X_train)

# Evaluation
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R-Squared Score:", r2_score(y_test, y_pred))
print("Predicted Prices:\n", y_pred[:5])

Mean Squared Error: 24.291119474973478
R-Squared Score: 0.6687594935356326
Predicted Prices:
[28.99672362 36.02556534 14.81694405 25.03197915 18.76987992]

import matplotlib.pyplot as plt

plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual Prices vs Predicted Prices")
plt.show()

```



```
# Plot actual values
plt.scatter(y_train,y_train_pred,color='blue', marker ='o',
label='Training data')
plt.scatter(y_test,y_test_pred,color='lightgreen', marker ='o',
label='Testing data')
# Labels and title
plt.xlabel("True Values")
plt.ylabel("Predicted")
plt.title("True Values vs Predicted")
plt.legend()
plt.show()
```

